Evaluation

Setup

Nineteen machines of 64 cores and 156 GiB: three signature verifiers, one coordinator, one sidecar, one load generator, and a YugabyteDB cluster of twelve tablet servers and three masters. Six of the twelve database machines also host a validator–committer and three others host a master, so no machine runs both. The generator embeds a mock ordering service and serves blocks to the sidecar over the Atomic Broadcast API, so these figures measure the validation and commit path with no ordering service in it. Transactions carry two read–write operations over 32 B keys, 262 B serialized, Ed25519-signed, in 10,000-transaction blocks. The published committer deployment is not the same shape — nine validator–committers on nine database nodes, dual 48-core servers with 64 GB — so what follows compares deployments, not matched hardware.

The end-to-end arm adds twenty machines of 32 cores and 78 GiB for the ordering service: four parties, each a router, a consenter, an assembler and four batchers, one per shard — so **four parties and four shards**. Routers, consenters and assemblers get a machine each; the sixteen batchers share eight, two per machine on separate ports. That pairing is measured rather than assumed: at two shards the service capped near 158,000 tps *per shard* whatever the batch size or decision interval, while batcher processes sat at 15 % to 20 % of a 32-core machine, so the constraint is per shard and the spare capacity for a second one was already on the box. Each batcher machine carries two 484 GB local disks, but both of its batchers currently write to the first, leaving the second unused.

The committer tier is unchanged across both arms, so a difference is attributable to ordering, and the two arms do not share hardware generation — the ordering machines have half the cores and half the memory of the committer machines.

A point is the highest offered rate that arrived in full, committed within 2 %, kept p99 under 1 s and held a flat inflight count, confirmed by a 300 s hold on a fresh deployment. Queue flatness matters as much as the latency bound: a rate can be committed in full out of a growing queue, which reports the queue's drain time as latency. Throughput counts transactions *finished*, committed plus rejected.

Committer throughput and tail latency

Throughput falls with transaction size (Fig. 1), 604,545 tps at one read–write operation to 274,364 at four: 55 % against the published 41 %. The mechanism is not the published one, which is lock contention in the coordinator's dependency graph: here both lock-wait histograms have a count rate of exactly zero, and the graph's cost is per key instead, about 420 ns each and flat from one key per transaction to eight.

Invalid signatures are close to free but never profitable: 558,364 tps with none, then 517,455, 519,455, 518,727 at 10 %, 20 %, 30 % — flat within 0.4 %, not the published 10 % *gain*. Driven instead at one common 559,872 tps, all three deliver it while the tail grows with the invalid share (1.1, 5.0, 7.5 s): rejection is cheap in throughput, expensive in latency.

Double spends do not reproduce, and a deployment setting is why. At the 120-way tablet pre-split this cluster uses, 0 % holds 518,000 tps and *no* rate qualifies at 5 % or above: about 20,000 tps at tens of seconds with every machine under 10 % CPU, so neither capacity nor a queue. At the default split the same shapes hold — 41,273, 30,000, 40,909 tps at 10 %, 20 %, 30 % and 334 ms to 533 ms p99, flat rather than declining — while costing the conflict-free case 1.8× (518,000 against 294,893). So the split is a workload-dependent trade, not a tuning win; narrowing the graph's chunk width refuted multi-key batching as the mechanism, leaving why 120 tablets collapse unresolved.

What latency costs at a given throughput

At 10,000-transaction blocks the median has a floor near 125 ms that does not fall with the rate, since nothing in a block moves until the block is cut (Fig. 2). At 500 transactions a block the floor is 40 ms and 380,100 tps holds at 125 ms median, 208 ms p99 — the median large blocks give at 10,000 tps, at thirty-eight times the rate. The top of that curve measures the generator, whose block preparation caps it near 850 blocks per second (425,000 tps here; 450,000 offered delivered 432,309), so the committer's ceiling at this size is not yet known.

End to end, with a real ordering service

TODO — running. The second arm puts a real Arma ordering service in the path. The published work has no end-to-end figure: ordering alone reaches 430,000 tps at four parties and four shards, and the committer alone 419,000–474,000, which bracket it. A size sweep is queued at 300 B, 512 B, 1,024 B, 2,048 B and 4,096 B. The published size sweep is a two-shard measurement, falling 413,000 to 27,000 tps over that range — each point within 10 % of 120 MB s^{-1}, so past 300 B it is bandwidth-bound and the rate is simply bytes per second over size. Its shape is therefore the comparable part, not its absolute values, since this arm runs four shards.

Threats to validity

The load generator is the instrument and twice became the limit: ECDSA signing, resolved with Ed25519, and block preparation, which caps the small-block curve. A knee is resolved to the search's 8 % step, so it is a lower bound, drawn one-sided in Fig. 1; and the baseline drifts 10 % between days, so only same-day comparisons are used.

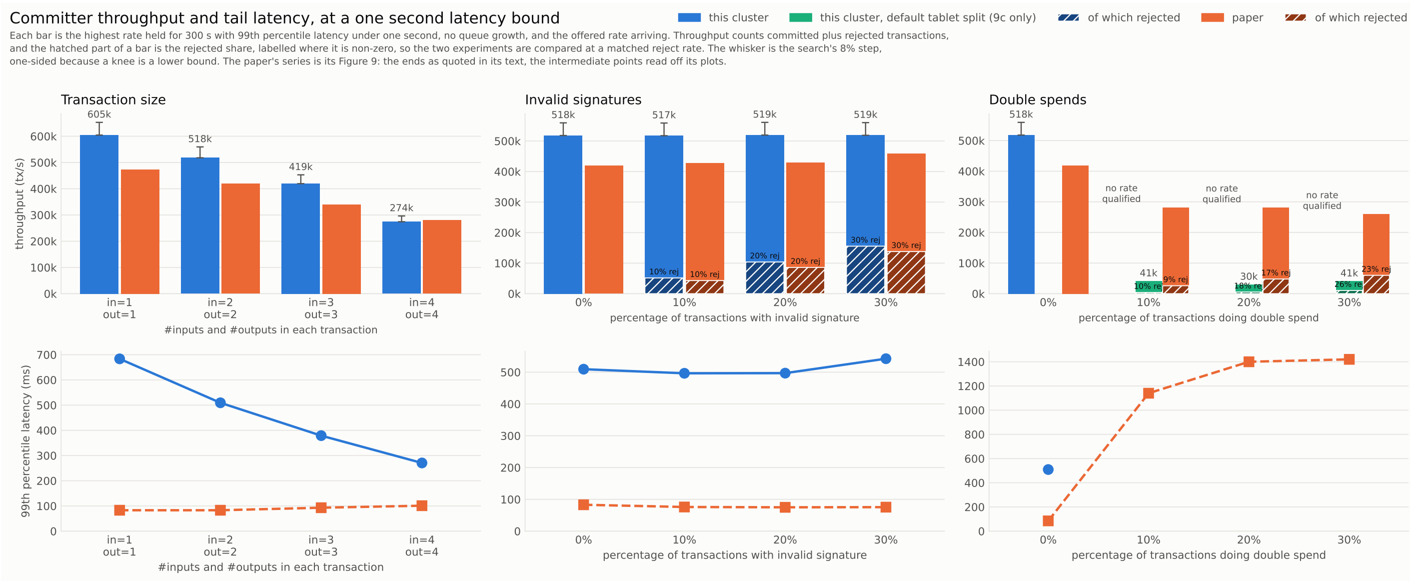

Figure 1: Throughput and 99th-percentile latency against transaction size, invalid-signature share and double-spend share, against the published figures. Hatched bars are the rejected share, so the two experiments are compared at matched reject rates rather than matched labels.

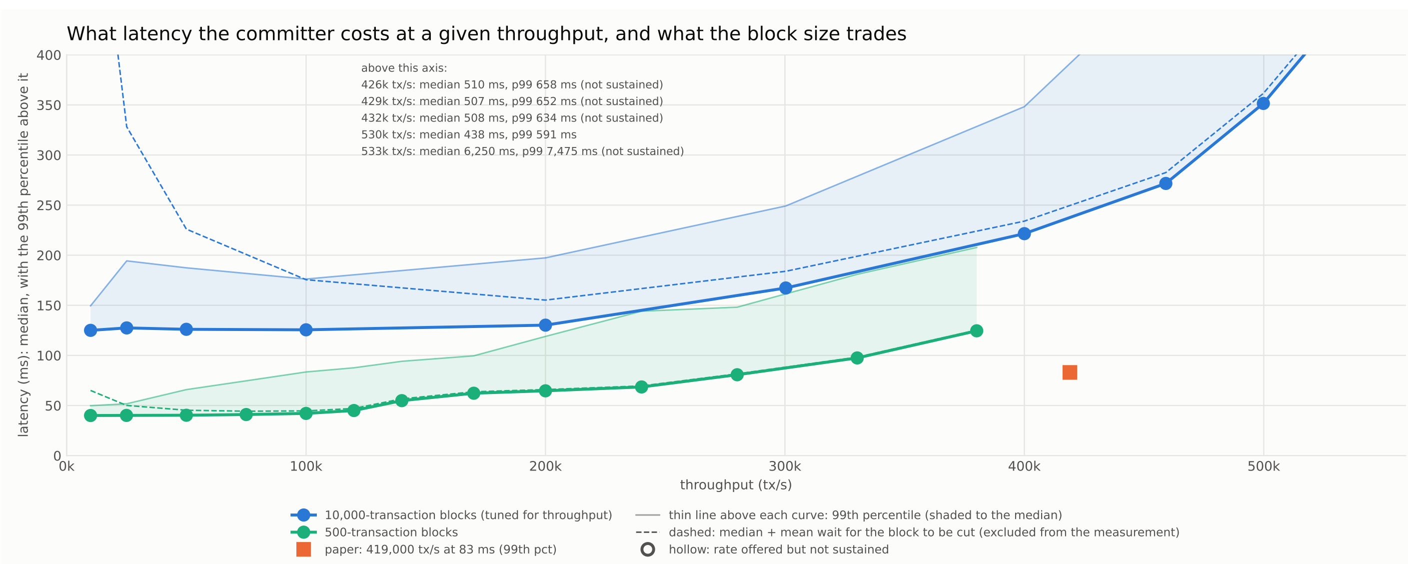

Figure 2: Latency against throughput at two block sizes; axis clipped at 400 ms, points above it labelled.